Exercise Title: "Bond Portfolio Risk Analyzer"

🧩 **Objective:**

Design and implement a small Java application that models a portfolio of financial instruments (specifically bonds), calculates basic risk metrics, and provides a simple API to query the results.

Rules of conduct:

1. The subject matter for this exercise is bonds. You are not expected to know anything in advance about bonds, but you are expected to be able to find the information you need and understand what you are doing.
2. The world of bonds is vast, and bonds come in many variants and flavours. For the purpose of this exercise, you may handle only the very simplest plain vanilla bond (e.g. a bond paying fixed periodic coupons without any additional conditions or stipulations). Supporting more complicated bond types is not required.
3. You are expected to use your AI coding assistant of choice. When you have completed this assessment, you are expected to hand in transcripts of you session/s with your AI assistant, as well as any context files you have created while working on this assessment.
4. We explain to our developers that while we promote the use of coding assistants, the human developer is ultimately responsible for all code entered into our codebases. We expect you to fully own the code you submit.

📋 **Requirements:**

1. Domain Modeling

- Create classes to represent:
    - Bond (with fields like ISIN, maturityDate, coupon dates, couponRate, faceValue, marketPrice)
    - Portfolio (a collection of bonds)

2. Risk Metrics

Implement methods to calculate:

- **Yield to Maturity (YTM)** for each bond (you may approximate)
- **Duration** and **Modified Duration**
- **Portfolio-level weighted average duration**

3. Data Input

- Implement a method to load data for multiple bonds. The method will accept JSON serialized into a string.

4. Interface

- Provide a REST API using Spring Boot, this rest API exposes all the interfaces discussed above

5. Design Expectations

- Use clean architecture principles (e.g., separation of concerns)
- Demonstrate use of design patterns
- Create tests that use the REST API and verify the results.

Please send your solution, including transcripts of your AI interactions and any generated context zipped back to francesca.lyle@ice.com. Alternatively, you may post your code to GitHub and share the link. We cannot accept submissions via any other publicly available file shares.